

图论与最短路径算法

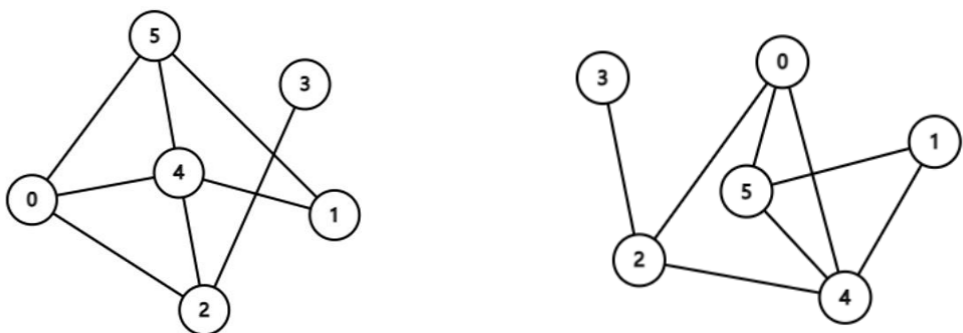
图

[图：在线直观图](#)

图的定义

对于平面上的**若干点**,把这些点用曲线或直线**连接起来**,**不考虑点的位置**与连线的曲直**长短**,这样形成的关系结构就是图.

例如以下两张图就是同一个图。（把节点抽象出来只剩下数据）

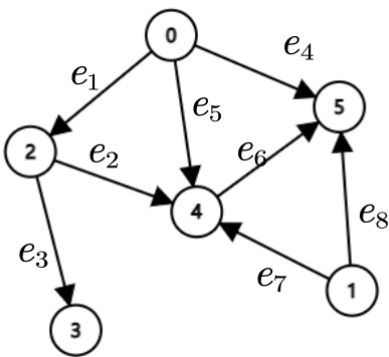


图的分类

- 有向图（节点之间用箭头连接）
- 无向图（节点之间用直线连接）（如上图所示）

图的表示方法

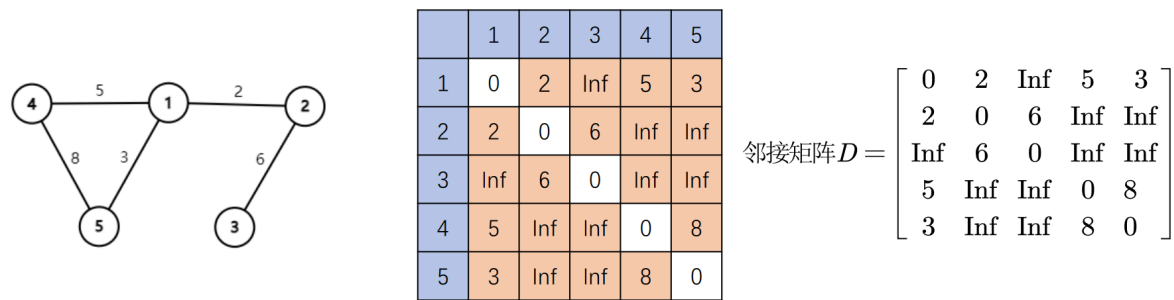
数学语言描述为 $G(V(G), E(G))$ 。 $V(vertex)$ 代表图的顶点集（顶点的集合）， $E(edge)$ 表示图的边集（边的集合）。



顶点集 $V = \{0, 1, 2, 3, 4, 5\}$ ，边集 $E = \{e_1, e_2, e_3, e_4, e_5, e_6, e_7, e_8\}$

图的数据结构

- 通过节点来建立方形矩阵（节点个数*节点个数）
- 矩阵之中的值的表示节点之间的距离
- 矩阵数值应该是关于 (n, n) 对称的
- 不确定的距离（没有线连接的节点之间）为 inf（无穷）

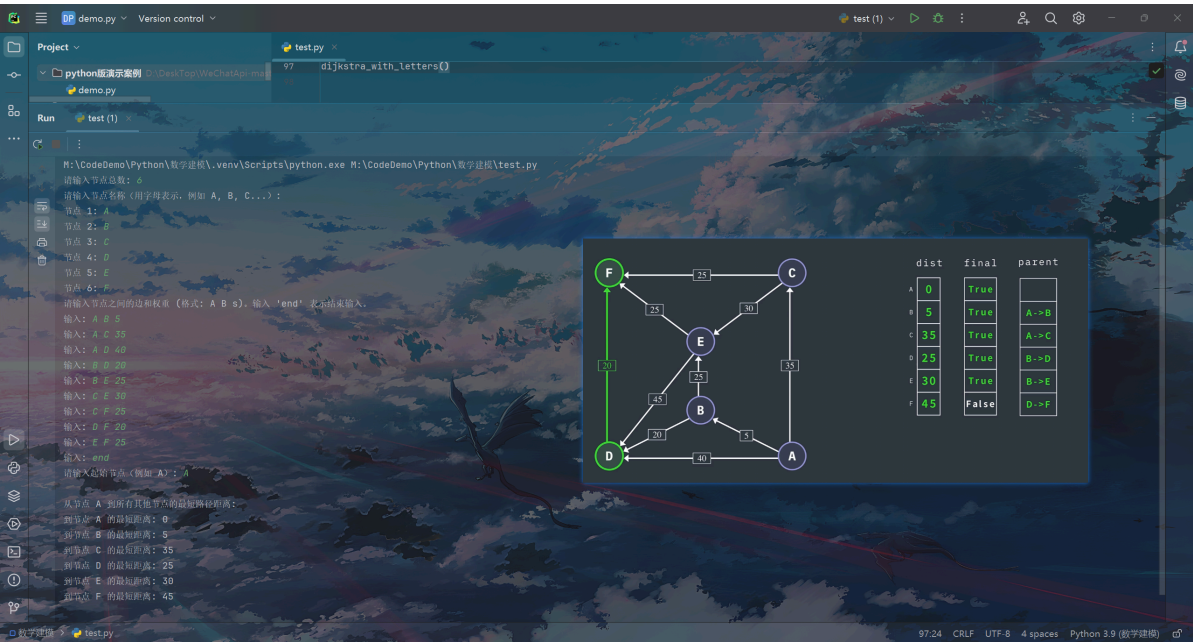


最短路径算法

迪杰斯特拉算法

思路参考B站视频：[最短路径 迪杰斯特拉 dijkstra算法 数据结构与算法](#)

代码参考ChatGPT



```
1 import heapq
2 from typing import Dict, List, Tuple
3
4
5 def dijkstra_with_letters():
6     """
7     使用 Dijkstra 算法计算最短路径，动态输入节点（使用字母表示）和边的信息。
8
9     输入：
10    1. 节点总数 `node_count`。
```

```

11 2. 节点名称（字母表示，例如 A, B, C...）。
12 3. 节点之间的边和权重，格式为：A B s（节点名称 + 权重），输入 'end' 表示结束。
13 4. 起始节点（字母表示）。
14
15 输出：
16 从起始节点到所有其他节点的最短路径距离。
17 """
18 # 输入节点总数
19 node_count = int(input("请输入节点总数: "))
20
21 # 输入节点名称
22 print("请输入节点名称（用字母表示，例如 A, B, C...）:")
23 nodes = [input(f"节点 {i + 1}: ").strip() for i in range(node_count)]
24
25 # 创建节点名称到索引的映射
26 node_to_index = {node: idx for idx, node in enumerate(nodes)}
27 index_to_node = {idx: node for idx, node in enumerate(nodes)}
28
29 # 初始化图的邻接表
30 graph: Dict[int, List[Tuple[int, int]]] = {i: [] for i in
range(node_count)}
31
32 # 动态输入边的信息
33 print("请输入节点之间的边和权重（格式：A B s）。输入 'end' 表示结束输入。")
34 while True:
35     edge = input("输入: ").strip()
36     if edge.lower() == "end":
37         break
38     try:
39         a, b, s = edge.split()
40         weight = int(s)
41         if a in node_to_index and b in node_to_index:
42             i, j = node_to_index[a], node_to_index[b]
43             graph[i].append((j, weight))
44         else:
45             print("节点名称不存在，请重新输入。")
46     except ValueError:
47         print("输入格式错误，请重新输入。")
48
49 # 输入起始节点
50 start_node = input("请输入起始节点（例如 A）: ").strip()
51 if start_node not in node_to_index:
52     print("起始节点不存在。")
53     return
54 start_index = node_to_index[start_node]
55
56 # 初始化最短距离字典
57 shortest_distances = {i: float('inf') for i in graph}
58 shortest_distances[start_index] = 0 # 起始节点到自身的距离为 0
59
60 # 最小堆（优先队列），存储（距离，节点索引）
61 priority_queue = [(0, start_index)]
62
63 # 记录已访问节点
64 visited = set()
65

```

```
66     # 核心 Dijkstra 算法
67     while priority_queue:
68         # 弹出距离最小的节点
69         current_distance, current_node = heapq.heappop(priority_queue)
70
71         # 如果节点已访问，跳过
72         if current_node in visited:
73             continue
74
75         # 标记节点为已访问
76         visited.add(current_node)
77
78         # 遍历当前节点的邻居
79         for neighbor, weight in graph[current_node]:
80             distance = current_distance + weight
81             # 如果找到更短路径，更新最短距离并加入队列
82             if distance < shortest_distances[neighbor]:
83                 shortest_distances[neighbor] = distance
84                 heapq.heappush(priority_queue, (distance, neighbor))
85
86     # 输出结果
87     print("\n从节点 {} 到所有其他节点的最短路径距离:".format(start_node))
88     for node_index, dist in shortest_distances.items():
89         node_name = index_to_node[node_index]
90         if dist == float('inf'):
91             print(f"节点 {node_name} 无法到达")
92         else:
93             print(f"到节点 {node_name} 的最短距离: {dist}")
94
95
96     # 调用函数
97     dijkstra_with_letters()
98
```